

1. 使用背景

在一些业务场景下（如业务模块二开等），业务模块需要自己开发代码满足具体需求。区别于目前已有的 `serverless/Action` 组件需要将业务模块代码上传到 `ESB` 的服务路径下以实现代码调用的方式，`serverless/Action` 跨服务 `action` 调用可以实现直接调用业务模块实现的 `dubbo` 接口，无须上传代码到 `ESB`。

2. 实现方案

业务模块实现 `ESB` 提供的接口，并将此接口按 `dubbo` 的方式注册到注册中心，然后通过 `ESB` 动作流配置实现接口调用。

2.1 业务模块（**provider**）

业务模块按以下步骤实现：

2.1.1 引入依赖

```
<dependency>
  <groupId>com.weaver</groupId>
  <artifactId>weaver-esb-server-api</artifactId>
</dependency>
```

2.1.2

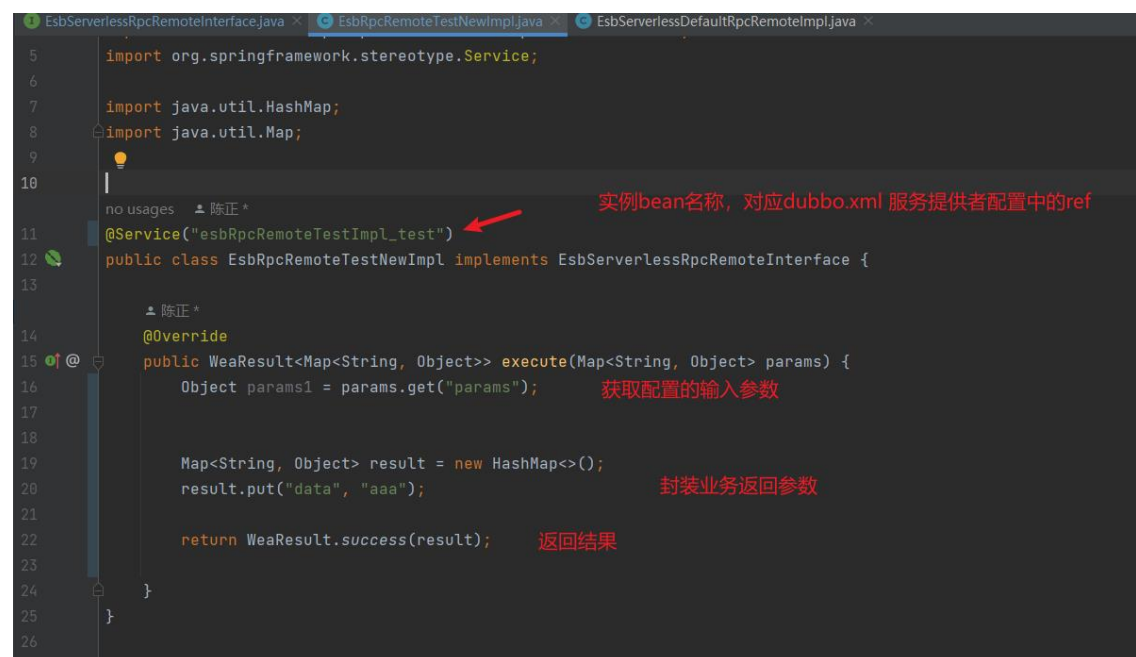
实现

`com.weaver.esb.api.rpc.EsbServerlessRpcRemoteInterface`

接口 （注意： 此处已调整新的 api， 不影响历史老接口调用， 新建的动作流默认使用 `com.weaver.esb.api.rpc.EsbServerlessRpcRemoteInterface` 接口， 如果想要使用老的 api 请在输入参数中添加 `useOldVersion (useOldVersion = 1)` 参数 ）

```
EsbServerlessRpcRemoteInterface.java x
1      package com.weaver.esb.api.rpc;
2
3      import com.weaver.common.base.entity.result.WeaResult;
4
5      import java.util.Map;
6
7      /**
8       * 动作流通用Dubbo调用远程实现接口
9       */
10     public interface EsbServerlessRpcRemoteInterface {
11
12         /**
13          * 业务模块实现此接口实现自己业务逻辑
14          * @param params 接口入参
15          * @return 执行结果
16          */
17         WeaResult<Map<String, Object>> execute(Map<String, Object> params);
18
19     }
```

例：



```
5 import org.springframework.stereotype.Service;
6
7 import java.util.HashMap;
8 import java.util.Map;
9
10
11 @Service("esbRpcRemoteTestImpl_test")
12 public class EsbRpcRemoteTestNewImpl implements EsbServerlessRpcRemoteInterface {
13
14     @Override
15     public WeaResult<Map<String, Object>> execute(Map<String, Object> params) {
16         Object params1 = params.get("params");
17
18         Map<String, Object> result = new HashMap<>();
19         result.put("data", "aaa");
20
21         return WeaResult.success(result);
22     }
23 }
24
25
26
```

2.1.3 发布服务

按照项目的架构体系，下面是通过 xml 文件去配置 dubbo 接口，发布对应服务。



```
<dubbo:service ref="esbRpcRemoteTestImpl_test"
interface="com.weaver.esb.api.rpc.EsbServerlessRpcRemoteInterface"
group="esb_test"
version = "esb_1.0"/>
```

ref ： 对应实现接口的 bean 实例名称。

interface :

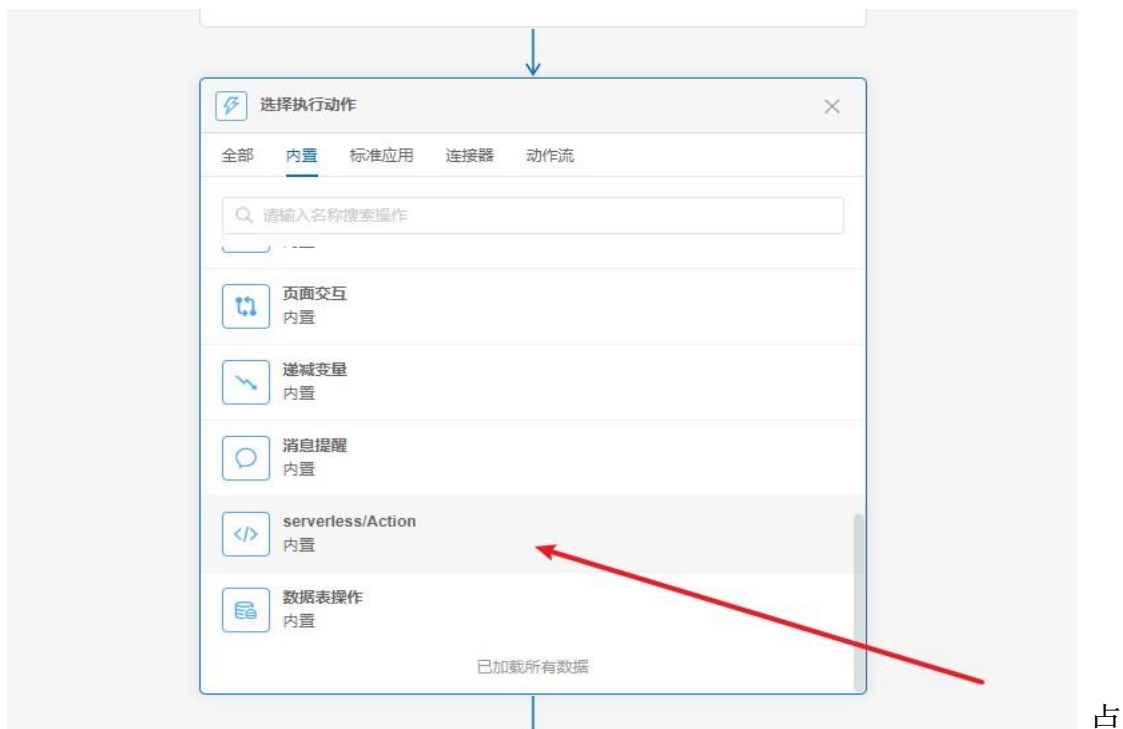
com.weaver.esb.api.rpc.EsbServerlessRpcRemoteInterface （固定不变）。

group: 用于区分不同实现的唯一约束。

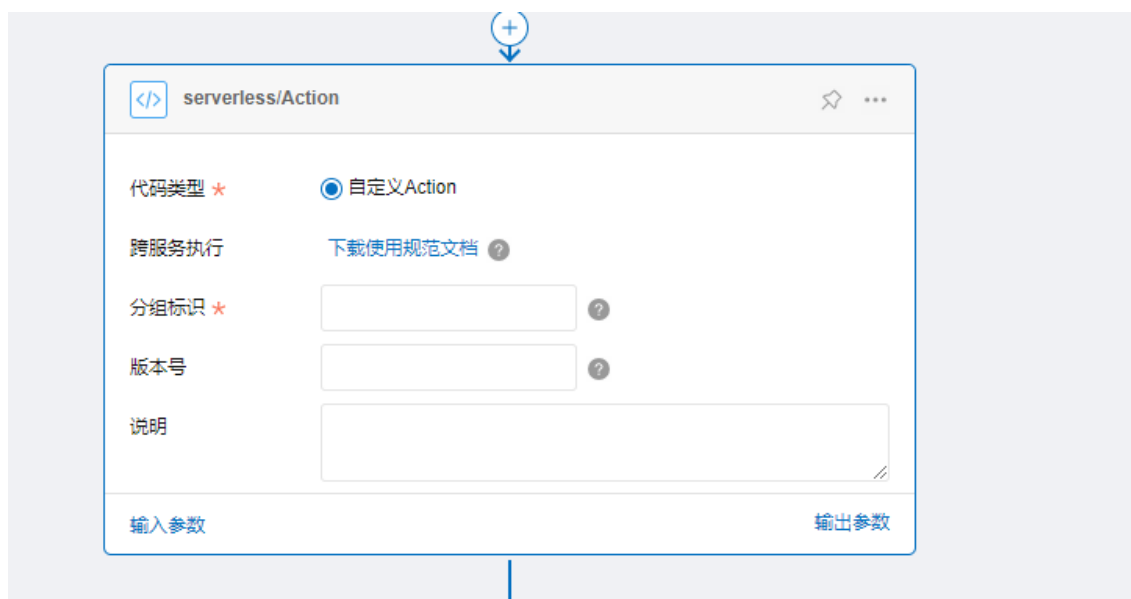
version: 可选参数，可用于后续多版本扩展，如果 dubbo 提供者配置了动作流也需要配置

3. ESB 动作流配置

【选择动作流】 - 【内置】 - 【serverless/Action】



填入发布服务时对应的分组标识（group）和版本号（version）
（和 dubbo 提供者配置文件的 group 和 version 对应，如果没有版本号就不用填）



配置输入参数和输出参数

输入参数配置

参数名称 *

显示名称

描述

参数类型 *

数据类型 *

必填

数组

动态赋值

aaa

aaa

aaa

文本

属性

固定值

哈哈哈

确定

取消

输入输出参数可在上下文中获取。

</>

内置: serverless/Action

请求数据

响应数据

参数名	字段类型
请求参数	Object
aaa ?	文本



所需配置简述：

输入参数：（调用接口所需要请求参数）

输出参数：（调用接口返回参数）

分组标识： 上述发布服务时填写的 group

版本号： 上述发布服务时填写的 version，没有可以不填，一般用于拓展使用。

4. 应用

4.1 通过 ecode 发布 dubbo 服务

可通过本地 IDE 开发（详情咨询二开模块），按照文档开始的规范进行代码编写

将代码 jar 包上传，以及配制文件编辑，然后重启二开服务，当 nacos 能查询到发布的服务之后 配置动作流进行调用

5. 常见问题

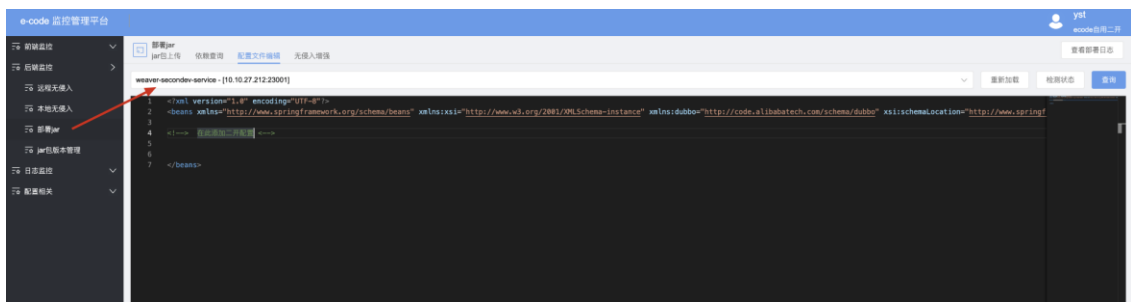
1. 实现代码需要放到 ESB 服务吗？

不需要，开发完成之后只需要在各自项目中注册 dubbo 服务，然后在动作流中根据发布服务时填写的 group 和 version（可选参数）去配置即可调用。

2. Duboo 发布服务在哪个配置文件？

根据项目架构体系来。E10 一般在项目 resources 目录下。

如果是二开服务二开修改 xml 统一到 e-code 监控管理平台里面改，访问地址 /ecode/monitor/loom/deploy/jar 配置文件编辑入口；如下图：



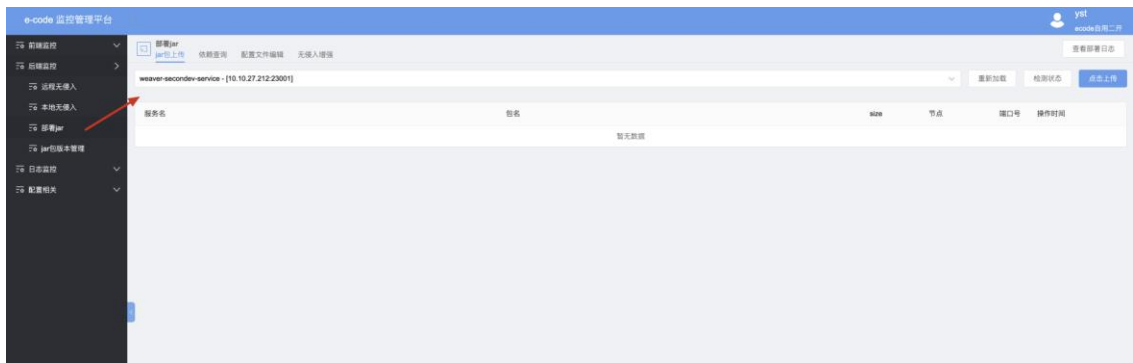
注意：

修改二开服务 xml 配置的时候，一定不要手动修改服务器上的 applicationContext-secondev-dubbo.xml 文件，此文件为标准文件，升级版本时会覆盖，会导致这部分二开功能全部失效！如果已经手动改了上述文件，那么需要将 xml 中的二开配置拷贝迁移到 e-code 里面，确保升级不会受到影响。

3. 后端如何开发代码，以及如何发布 jar 包？

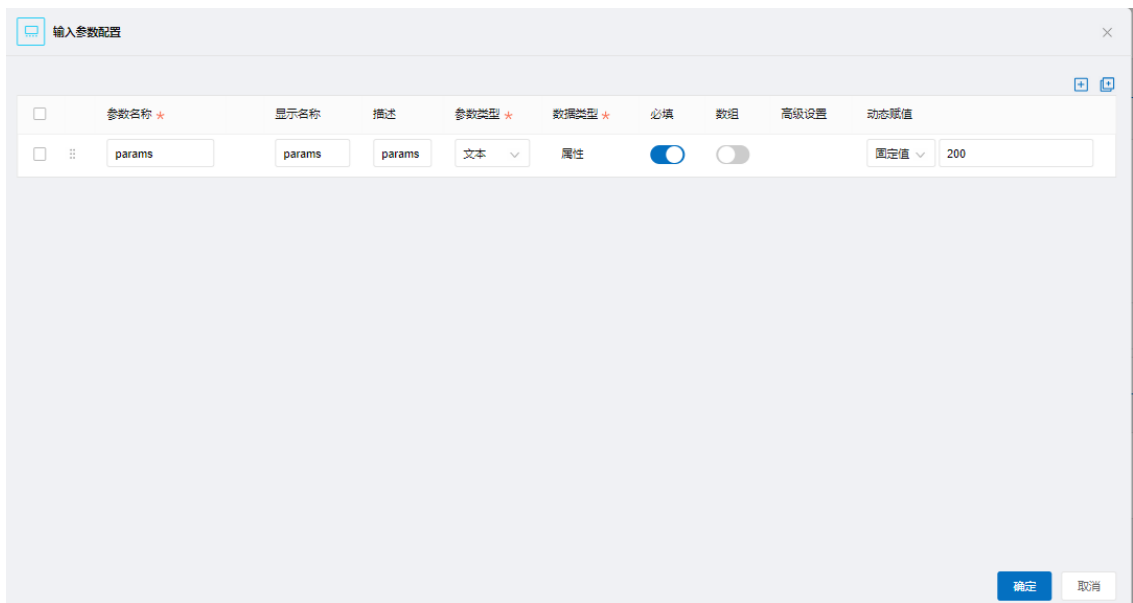
后端 Java 开发请走本地开发模式。参考文档：<https://www.e-cology.com.cn/sp/doc/docDetail/8140901019737699195>

开发好打 jar 上传部署到二开服务，如下图

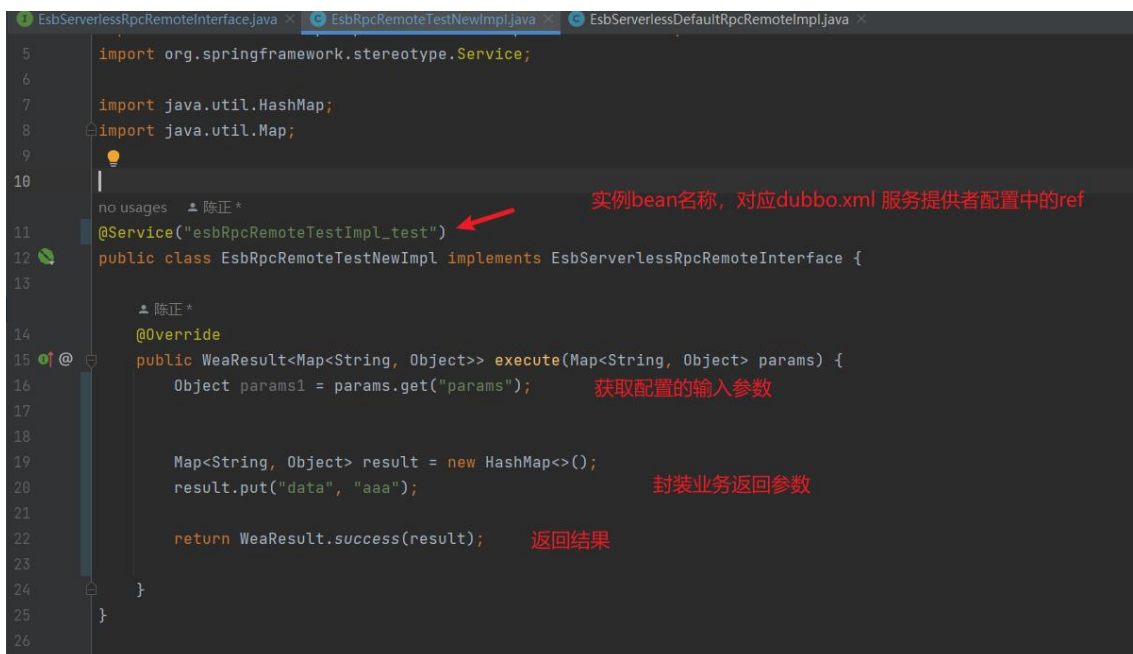


4. 输入参数代码如何获取？

例 如下图，如果在动作流中配置 params 的参数 key



则按照下面图中方法获取



5. 如何确认发布的服务是否生效

可在 dubbo 对应注册中心中查看，按照当前架构体系，一般使用 nacos 注册中心，则在如下图所示地方查看。



6. 常见报错排查方案

```
1030000007037720000 : {
  "batchKey": "868973503008595977",
  "componentId": "842644833797816346",
  "componentName": "内置: serverless/Action",
  "componentType": "Serverless/Action",
  "conditionLog": {},
  "costTime": 78,
  "employeeId": "6491435406850069654",
  "endTime": "2023-05-31 17:01:00.668",
  "exceptionHandlerLog": {},
  "executeStatus": "异常",
  "multipleLogList": {},
  "parallelLog": {},
  "requestData": "{}",
  "response": {
    "context": "{\\\"ExecuteStatus\\\":{\\\"formStatus\\\":{\\\"description\\\":\\\"ServerlessAction发生异常, 失败原因: \\\"exception\\\":\\\"com.weaver.framework.remote.common.exception.RemoteException: Microservice rest rpc Que\\\"message\\\":\\\"ServerlessAction发生异常, 失败原因: Microservice rest rpc Query service name is empty\\\",\\\"status\\\":\\\"EXCEPTION\\\"",
    "exception": "com.weaver.framework.remote.common.exception.RemoteException: Microservice rest rpc Que",
    "message": "ServerlessAction发生异常, 失败原因: Microservice rest rpc Query service name is empty",
    "status": "EXCEPTION"
  },
  "startTime": "2023-05-31 17:01:00.590"
}
```

该报错一般是 dubbo 提供者未成功发布，检查下 dubbo 服务的 group 和 version 动作流配置是否正确，并且可以在 nacos 查询对应的服务有没有发布成功

```

},
"1666318463582474390": {
  "batchKey": "874439076652711937",
  "componentId": "871519198993932292",
  "componentName": "serverless/Action",
  "componentType": "Serverless/Action",
  "conditionLog": {},
  "costTime": 38,
  "employeeId": "6491435630875954392",
  "endTime": "2023-06-15 10:30:13.564",
  "exceptionHandlerLog": {},
  "executeStatus": "异常",
  "multipleLogList": [],
  "parallelLog": {},
  "requestData": "{}",
  "response": {
    "context": "{\\\"ExecuteStatus\\\":\\\"{\\\"formStatus\\\":\\\"{\\\"description\\\":\\\"ServerlessAction发生异常, 失败原因:\\",
    "exception": "com.alibaba.fastjson.JSONException: safeMode not support autoType : org.springframework.",
    "message": "ServerlessAction发生异常, 失败原因: safeMode not support autoType : org.springframework.",
    "status": "EXCEPTION"
  },
  "startTime": "2023-06-15 10:30:13.526"
}
}

```

```

17   tenantKey : t/akvdnt84
18 },
19 "841674046547034114": {
20   "batchKey": "870848393735356416",
21   "componentId": "841674050875613349",
22   "componentName": "serverless/Action",
23   "componentType": "Serverless/Action",
24   "conditionLog": {},
25   "costTime": 60107,
26   "employeeId": "6491435568718952946",
27   "endTime": "2023-06-05 18:17:32.508",
28   "exceptionHandlerLog": {},
29   "executeStatus": "异常",
30   "multipleLogList": [],
31   "parallelLog": {},
32   "requestData": "{}",
33   "response": {
34     "context": "{\\\"ExecuteStatus\\\":\\\"{\\\"formStatus\\\":\\\"{\\\"description\\\":\\\"ServerlessAction发生异常, 失败原因:\\",
35     "exception": "com.weaver.framework.remote.common.exception.RemoteException: Rest Rpc execute is fail,
36     "message": "ServerlessAction发生异常, 失败原因: Rest Rpc execute is fail,url->[http://WEAVER-SUB-PREF
37     "status": "EXCEPTION"
38   },
39   "startTime": "2023-06-05 18:16:32.401"
40 }
41 }

```

该报错一般是提供者有异常，可在提供者 err.log 日志中查询错误信息，部分异常信息根据提供者业务不同，失败原因中报错信息也不同

7. 接口超时时间如何修改

修改 dubbo 接口超时时间

```

version = "esb_1.0"/>
<dubbo:service ref="esbRpcRemoteTestImpl_test_new"
  interface="com.weaver.esb.api.rpc.EsbServerlessRpcRemoteInterface"
  group="esb_test_new" timeout="300000"/>

```

修改 naocs 配置文件 `weaver-esb-server-service.properties` （部分合体项目可能需要到指定合体服务的配置文件去修改）

添加如下配置, 超时时间自定义, 单位毫秒

```
weaver.rest.web.restRpc.[group/dubboapi 全路径  
/version].restProperties.connectTimeOut=30000
```

```
weaver.rest.web.restRpc.[group/dubboapi 全路径  
/version].restProperties.writeTimeOut=60000
```

```
weaver.rest.web.restRpc.[group/dubboapi 全路径  
/version].restProperties.readTimeOut=3000
```

例：如 `group :esb_test_new` , `version : seconddev1.0`

```
weaver.rest.web.restRpc.[esb_test_new/  
com.weaver.esb.api.rpc.EsbServerlessRpcRemoteInterface/seconddev1.0].restProperties.c  
onnectTimeOut =xxx
```

```
weaver.rest.web.restRpc.[esb_test_new/  
com.weaver.esb.api.rpc.EsbServerlessRpcRemoteInterface/seconddev1.0].restProperties.  
writeTimeOut =xxx
```

```
weaver.rest.web.restRpc.[esb_test_new/  
com.weaver.esb.api.rpc.EsbServerlessRpcRemoteInterface/seconddev1.0].restProperties.r  
eadTimeOut =xxx
```

修改完成后重启 `esb` 所在服务